

Open Spec: Architektura Agregatora Nieruchomości

Podejście minimalistyczne (Python + SQLite) | Data dokumentu: 27 Lipca 2026

1. Cel i Kontekst

Celem projektu jest refaktoryzacja obecnego systemu pobierania ogłoszeń nieruchomości do modelu **ELT (Extract, Load, Transform)**. Zmiana ta eliminuje problem zawodnych filtrów po stronie portali ogłoszeniowych poprzez szerokie pobieranie surowych danych i przeniesienie logiki filtrowania, normalizacji i deduplikacji do warstwy analitycznej.

Zgodnie z założeniami, system wykorzystuje minimalistyczny i bezserwerowy (serverless-like) stos technologiczny: **Python** do orkiestracji i pobierania oraz **SQLite** jako silnik bazy danych i transformacji (wykorzystując wbudowane wsparcie dla JSON1).

2. Architektura Systemu

Proces jest podzielony na wyraźne warstwy, analogiczne do klasycznej architektury *Medallion Architecture* (Bronze, Silver, Gold), ale zaimplementowane w pojedynczym pliku bazy SQLite.

2.1. Warstwa Ekstrakcji (Python)

Scrapery w Pythonie (np. `requests`, `httpx` lub `playwright`) wykonują minimalnie sparametryzowane zapytania do portali (np. "cała Warszawa"). Nie parsują danych szczegółowo. Wyciągają jedynie surowy payload (najlepiej JSON z tagów `<script>` w HTML lub bezpośrednio z ukrytych API).

2.2. Warstwa Bronze (Surowe Dane)

Python zapisuje surowe odpowiedzi bezpośrednio do tabeli w SQLite. Brak weryfikacji typów czy struktury na tym etapie pozwala na maksymalną odporność na błędy (resilience) scrapera.

```
CREATE TABLE bronze_listings (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  source_portal TEXT NOT NULL,  
  raw_payload JSON NOT NULL,  
  scraped_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

2.3. Warstwa Silver (Normalizacja i Filtracja)

Wykorzystując rozszerzenie JSON1 w SQLite (dostępne natywnie), dane są wyciągane z surowego payloadu, rzutowane na odpowiednie typy, i tu aplikowane są filtry biznesowe.

```
-- Widok aplikujący filtry na poziomie bazy, a nie scrapera
CREATE VIEW silver_listings AS
SELECT
    id AS raw_id,
    source_portal,
    json_extract(raw_payload, '$.id') AS source_id,
    json_extract(raw_payload, '$.price') AS price_pln,
    json_extract(raw_payload, '$.area') AS area_m2,
    json_extract(raw_payload, '$.rooms') AS rooms,
    json_extract(raw_payload, '$.location.lat') AS lat,
    json_extract(raw_payload, '$.location.lon') AS lon,
    scraped_at
FROM bronze_listings
WHERE
    price_pln IS NOT NULL
    AND area_m2 > 0;
```

2.4. Warstwa Gold (Deduplikacja i RCIWN)

Ponieważ te same mieszkania trafiają na wiele portali, konieczne jest wygenerowanie hasha deduplikacyjnego. SQLite nie ma rozbudowanych funkcji przestrzennych (chyba że z użyciem SpatiaLite), ale można to aproksymować, zaokrąglając współrzędne (np. do 3 miejsc po przecinku co daje dokładność ok. 100m).

```
-- Widok deduplikujący fizyczne lokale
CREATE VIEW gold_unique_properties AS
SELECT
    -- Klucz deduplikacyjny: zaokrąglone współrzędne + metraż + liczba pokoi
    round(lat, 3) || '_' || round(lon, 3) || '_' || CAST(area_m2 AS INTEGER) AS property_hash,
    MIN(price_pln) AS lowest_current_price,
    area_m2,
    rooms,
    COUNT(*) AS appearance_count,
    MAX(scraped_at) AS last_seen
FROM silver_listings
GROUP BY 1, 3, 4;
```

3. Integracja z Rejestrem Cen (RCiWN)

Podjęcie minimalistyczne do geokodowania: Rejestr Cen i Wartości Nieruchomości z Warszawy nie operuje na precyzyjnych współrzędnych, lecz na obrębach i adresach.

Zamiast instalować ciężkie silniki GIS, proponuje się następujący flow w Pythonie (uruchamiany jako część zasilania):

1. Pobierz listę transakcji RCIWN (np. z pliku CSV) i załaduj do tabeli `rciwn_transactions`.
2. W Pythonie, wykorzystaj lekką bibliotekę `geopy` (Nominatim) lub statyczny słownik ulic do przetworzenia danych adresowych.
3. Dołącz metryki referencyjne w SQL:

```
CREATE VIEW gold_properties_vs_market AS
SELECT
    p.property_hash,
    p.lowest_current_price,
    p.area_m2,
    (p.lowest_current_price / p.area_m2) AS price_per_sqm_offer,
    r.avg_price_per_sqm AS price_per_sqm_market,
    ((p.lowest_current_price / p.area_m2) / r.avg_price_per_sqm) - 1 AS overvaluation_pct
FROM gold_unique_properties p
LEFT JOIN rciwn_aggregates r
    ON p.district = r.district
WHERE p.last_seen >= date('now', '-1 day');
```

4. Orkiestracja i Działanie (Flow)

Całość systemu może być zamknięta w jednym repozytorium uruchamianym z poziomu `cron` lub `systemd` na pojedynczej maszynie:

- `01_extract.py` - Uruchamia żądania HTTP i ładuje surowy JSON poprzez `INSERT` do `bronze_listings`.
- `02_transform.py` - Uruchamia predefiniowane skrypty `.sql` tworzące/odświeżające dane w warstwach Silver/Gold (np. odświeżanie tabel zmaterializowanych).
- `03_report.py` - Odpytuje warstwę Gold, formatuje dane i dystrybuje poranny raport (np. e-mail).